

Secure Rhythm Complete Project Explanation

1) Project Purpose and Core Idea

Secure Rhythm is a behavioral biometrics authentication platform. Instead of trusting only static credentials, it analyzes *how a person types* (keystroke dynamics) and uses that behavior as an identity signal. The system measures dwell time (how long keys are held), flight time (time between key events), typing speed, consistency, and prompt-match quality. It then decides whether to continue with OTP, ask for fallback password verification, or deny access.

This project is split into two major applications:

- Frontend: React + TypeScript + Vite dashboard and live demo UI.
- Backend: Node.js + Express + MongoDB API with OTP, profile management, analytics, and a dataset-driven ML layer.

The most important design decision is that authentication is *risk-based*:

- High-quality behavioral match does not immediately finalize access.
- OTP is still required for final approval.
- Low-confidence sessions switch to fallback password, then OTP.

That makes it a layered authentication flow rather than a single binary check.

2) Monorepo Structure and Responsibilities

```
secure-rhythm/  
  src/                # Frontend app (React)  
  backend/src/        # Backend app (Express)  
    config/           # env + database bootstrapping  
    controllers/      # HTTP handlers  
    routes/           # Route definitions  
    services/         # Business logic  
    models/           # Mongoose schemas  
    middleware/       # auth/error/notFound  
    validation/       # Zod request validation  
    utils/            # shared helpers
```

Explanation:

- controllers are thin: parse/validate input and call services.
- services hold real logic (auth, OTP, analytics, biometric scoring).
- models define persisted objects for users, profiles, sessions, OTP challenges.
- validation prevents malformed payloads from entering business logic.

3) Runtime and Startup Flow

When backend starts, it does more than just open a port.

```
// backend/src/server.js  
await connectToDatabase();  
await ensureMLModelReady();  
await ensureDefaultProfiles();  
app.listen(env.port);
```

Explanation:

- MongoDB connection is required first.
- ML artifact is loaded (or trained from dataset if missing).
- Default profile records are seeded/updated.
- Server starts only after these prerequisites are valid.

This guarantees a working authentication baseline at boot time.

4) Frontend Architecture (React App)

Frontend entry and routing are centralized in `src/App.tsx`.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Index />} />
    <Route path="/graph" element={<GraphPage />} />
    <Route path="/analysis" element={<AnalysisPage />} />
    <Route path="/auth" element={<AuthPage />} />
    <Route path="/add-user" element={<AddUserPage />} />
    <Route path="/sessions" element={<SessionsPage />} />
  </Routes>
</BrowserRouter>
```

Explanation:

- `/` is the live typing authentication experience.
- `/auth` handles account signup/login.
- `/add-user` creates biometric user profiles.
- `/sessions` shows private history summary for authenticated account.
- `/graph` and `/analysis` render operational and dataset analytics.

The app wraps routes in `AuthProvider` and `QueryClientProvider` so auth state and API caching are globally available.

5) API Client Pattern in Frontend

`src/lib/api.ts` is a strongly-typed API layer.

```
const response = await fetch(getUrl(path), {
  method,
  headers: {
    "Content-Type": "application/json",
    Authorization: authToken ? `Bearer ${authToken}` : undefined,
  },
  body: JSON.stringify(payload),
});
```

Explanation:

- Single request wrapper centralizes token handling and error parsing.
- TypeScript interfaces define payload and response contracts.
- UI pages call typed functions (`identifyTyping`, `verifyOtp`, `fetchSessionSummary`, etc.) instead of manual fetch code.

This keeps page components focused on state and rendering.

6) Authentication Domain Model (Important Separation)

There are two different "user" concepts:

1. **AppUser** (backend/src/models/AppUser.js)
 - Real account that logs into dashboard with JWT.
 - Fields: fullName, email, passwordHash, active, timestamps.
2. **UserProfile** (backend/src/models/UserProfile.js)
 - Biometric identity profile used for typing recognition.
 - Fields: name, email, avgDwell, avgFlight, tolerance, fallbackPasswordHash, active.

Why this matters:

- A signed-in account (AppUser) can create/view/own demo sessions.
- The typing identity being detected can be any profile in the biometric dataset.

7) Login, Signup, and Username-to-Email Resolution

accountService supports email or username input.

```
// backend/src/services/accountService.js
if (isEmailAddress(loginIdentifier)) {
  user = await AppUser.findOne({ email: loginIdentifier, active: true });
} else {
  const matches = await AppUser.find({
    active: true,
    email: { $regex: `^${escapeRegExp(loginIdentifier)}@`, $options: "i" },
  }).limit(2);
  // 0 => invalid; 1 => use it; >1 => ask for full email
}
```

Explanation:

- If user enters full email: direct lookup.
- If user enters username: backend searches DB by email local-part prefix.
- If multiple matches exist for same username across domains, backend blocks ambiguity and asks for full email.

Signup also normalizes email or username input using `normalizeEmailOrUsername(...)`.

8) Typing Biometrics Pipeline (Core Logic)

`identifyTypingPattern` in `biometricsService` is the main flow.

```
const featureMetrics = calculateFeatureMetrics({
  typedText, promptText, dwellTimes, flightTimes, keyCount, backspaceCount,
});

const mlPrediction = await predictWithMlModel({
  avgDwell, avgFlight, dwellTimes, flightTimes, featureMetrics,
});
```

```
const { bestProfile, confidence, uncertainty } = calculateMatch(...);
const canDirectlyAuthenticate =
  confidence >= env.authConfidenceThreshold &&
  featureMetrics.promptMatchRatio >= env.promptMatchThreshold &&
  featureMetrics.tenSecondMatch;
```

Explanation:

- Feature extraction computes behavioral and quality metrics.
- Two model signals are merged:
- Hybrid behavioral scorer (modelService).
- Dataset artifact predictor (mlModelService).
- Confidence and uncertainty are combined.
- Even with strong match, result becomes otp_pending (not immediate success).

9) Hybrid Scoring Internals

modelService has weighted scoring blocks:

- Distance score: how close dwell/flight/speed are to profile baseline.
- Behavior score: prompt match + consistency + backspace behavior.
- Temporal score: 10-second window stability.

```
const score = 0.55 * distanceScore + 0.25 * behaviorScore + 0.2 * temporalScore;
const mc = monteCarloDropout({ distanceScore, behaviorScore, temporalScore });
return { confidence: mc.adjustedConfidence, uncertainty: mc.uncertainty };
```

Explanation:

- Monte Carlo dropout-style perturbation simulates uncertainty around confidence.
- Higher uncertainty pushes warning messages and fallback recommendations.
- This is a practical risk signal, not only a raw classification output.

10) OTP Generation, Storage, and Delivery

OTP service creates secure challenge records in MongoDB.

```
const challenge = await OtpChallenge.create({
  profileId: profile._id,
  sessionId,
  purpose,
  codeHash: sha256(otpCode),
  attempts: 0,
  maxAttempts: env.otpMaxAttempts,
  expiresAt,
});
```

Explanation:

- Only hash is stored (codeHash), never plaintext OTP.
- expiresAt plus TTL index auto-removes stale OTP docs.
- Attempts are counted server-side and access is denied after max attempts.

OTP email destination behavior

This project now resolves email from DB for the matched username/profile.

```
const destinationEmail = String(profile.email ?? "").trim().toLowerCase();
if (!isEmailAddress(destinationEmail)) {
  throw new HttpError(400, "Selected username does not have a valid email configured in DB for OTP");
}
await sendOtpMail({ to: destinationEmail, ... });
```

Explanation:

- OTP is sent to the stored `UserProfile.email`.
- If email is not configured for that profile, OTP send is intentionally blocked.
- This avoids accidentally sending OTP to guessed/generated addresses.

11) Fallback Password + OTP Second Stage

If typing confidence is low, `verifyFallbackPassword` compares typed password with profile fallback hash.

```
const matches = await bcrypt.compare(password, userProfile.fallbackPasswordHash);
const authStatus = matches ? "otp_pending" : "denied";
```

Explanation:

- Successful fallback does not finalize authentication.
- It still requires OTP to complete final auth.
- Failed fallback immediately sets session to denied.

12) Session Tracking and Analytics

Every attempt is persisted as `TypingSession` with rich metrics.

Stored fields include:

- raw timings (`dwellingTimes`, `flightTimes`)
- aggregate timings (`avgDwell`, `avgFlight`)
- operational metrics (`wpm`, `consistencyScore`, `promptMatchRatio`, etc.)
- auth lifecycle (`authStatus`, `authMethod`, fallback flags, warnings)
- owner (`createdBy`) for private session views.

`/api/sessions/summary` uses MongoDB aggregation to compute dashboard stats such as:

- success rate
- average confidence
- avg typing speed and quality metrics
- counts of authenticated/fallback/denied/otp_pending

13) DSL Dataset Analysis Engine

`dslAnalysisService` reads full CSV from `DSL_DATASET_PATH`, parses timing features, and computes:

- subject/session/repetition distributions
- hold-latency averages

- feature means
- histograms (H.period, DD.period.t, UD.period.t)
- negative value frequency by feature
- sampled scatter points

This powers the 10 chart views on /analysis.

14) ML Model Artifact Layer

mLModelService builds and caches a JSON model artifact at ML_MODEL_PATH.

Key idea:

- It engineers feature vectors from dwell/flight values.
- Builds two encoders (CNN-style and Bi-LSTM-style embeddings).
- Uses nearest-neighbor style classification on embeddings.
- Fuses predictions into an ensemble.
- Produces train/validation and report history.

Runtime prediction (predictWithMLModel) returns:

- top label probability
- per-label probabilities
- uncertainty score based on entropy + model disagreement.

15) End-to-End Workflow (Understandable Step Sequence)

A) Live typing authentication path

1. User types sentence on Index page.
2. Frontend records keydown/keyup timings.
3. Frontend calls POST /api/auth/identify with timing arrays + metadata.
4. Backend extracts features, scores profiles, computes confidence/uncertainty.
5. Backend saves session.
6. If score quality is strong: backend creates OTP challenge and emails OTP.
7. Frontend shows OTP panel and masked destination.
8. User submits OTP to POST /api/auth/verify-otp.
9. If OTP valid: session status becomes authenticated.

B) Fallback path

1. Typing result is low confidence -> fallback.
2. User enters fallback password.
3. Backend verifies profile fallback hash.
4. If correct -> OTP challenge created and sent to profile email in DB.

5. User verifies OTP -> final authentication.

C) Password-only path

1. User enters profile name + password without typing flow.
2. Backend creates session with `authMethod = password-only`.
3. OTP still required to complete secure auth.

16) Security Controls Present in Project

- Password and fallback secrets hashed via `bcrypt`.
- OTP stored hashed; plaintext never persisted.
- OTP expiration and attempt limits enforced.
- JWT-based API auth for protected endpoints.
- Zod validation on incoming payloads.
- CORS allow-list from env.
- Error middleware returns consistent sanitized responses.

17) Environment Variables You Must Configure

From backend/.env.example:

```
MONGO_URI=mongodb://127.0.0.1:27017/behavioural_biometrics
JWT_SECRET=change_this_to_a_long_random_secret
SMTP_HOST=smtp.gmail.com
SMTP_USER=your_mail@example.com
SMTP_PASS=your_app_password
SMTP_FROM=KeystrokeID <your_mail@example.com>
ML_MODEL_PATH=C:/.../backend/data/keystroke-model.json
PROFILE_EMAIL_DOMAIN=keystrokeid.local
DSL_DATASET_PATH=C:/Users/Deepa/Downloads/DSL-StrongPasswordData.csv
```

Explanation:

- `SMTP_*` must be correct for real OTP email delivery.
- `DSL_DATASET_PATH` must point to existing CSV.
- `ML_MODEL_PATH` must be writable so model artifact can be persisted.

18) Practical Request Flow Example

```
POST /api/auth/fallback
{
  "detectedUser": "Varsha",
  "password": "secure123"
}
```

Backend behavior:

- Finds `UserProfile(name="Varsha")`.
- Validates fallback password hash.
- Reads `profile.email` from DB.

- Creates OTP challenge row and sends OTP to that stored email.
- Returns otpChallengeId, otpDestination, and authStatus: otp_pending.

Then client submits OTP:

```
POST /api/auth/verify-otp
{
  "challengeId": "<challenge_object_id>",
  "otpCode": "123456"
}
```

If valid:

- Session is updated to authenticated.
- Confidence set to 100.

19) Frontend Page Responsibilities Summary

- Index.tsx: live typing capture, identification, fallback password UI, OTP verification UI.
- AuthPage.tsx: signup/login account (email or username allowed).
- AddUserPage.tsx: create biometric profile with timing baseline and fallback password.
- SessionsPage.tsx: authenticated account's personal session history and aggregate summary.
- GraphPage.tsx: live operational dashboards from /api/analytics/charts.
- AnalysisPage.tsx: dataset-level DSL insights + model report views.

20) What Makes This Project Strong

- Multi-layer auth (behavior + password fallback + OTP).
- Full observability (sessions, warnings, trends, confidence curves).
- Dataset analytics and model-report visibility inside product UI.
- Clear service layering and typed frontend API integration.

21) Current Constraints and Improvement Opportunities

- Model implementation is deterministic engineering + kNN-style embeddings, not a heavy deep-learning runtime server.
- SMTP errors currently depend on environment quality; add richer delivery audit table if needed.
- Username lookup in account login can be ambiguous for same local-part across domains; currently handled by explicit 409 conflict.
- Could add rate limiting on auth routes for brute-force resistance.

22) Quick Setup and Run Checklist

1. cd secure-rhythm/backend && npm install
2. Create backend/.env from .env.example
3. Ensure MongoDB is running
4. npm run dev (backend)

5. `cd ../ && npm install`

6. `npm run dev (frontend)`

7. Open `http://localhost:8080`

You now get:

- live typing authentication
- OTP routing to profile emails from DB
- session history and analytics dashboards
- DSL dataset analysis and model report views